

Backend — API GraphQL

API GraphQL desplegada en el edge con **Cloudflare Workers**, schema code-first con **Pothos**, ORM **Prisma 7** sobre **PostgreSQL/Supabase** y autenticación con JWT de Supabase verificado mediante HMAC-SHA256.

Comandos principales

```
npm install
npx prisma generate      # Genera el cliente Prisma
npx wrangler dev         # Worker local → http://localhost:8787/graphql
npm run dev              # tsx watch (sin Wrangler)
npm test                 # Tests con Vitest
npm run deploy           # Deploy a Cloudflare Workers
npm run cf-typegen       # Regenera tipos de Cloudflare
```

Stack

Capa	Tecnología	Detalle
Runtime	Cloudflare Workers	Edge, serverless, global
API	GraphQL Yoga	Server GraphQL compatible Workers
Schema	Pothos	Code-first, plugins Prisma + Relay
ORM	Prisma 7	Type-safe, con Prisma Accelerate
Base de datos	PostgreSQL / Supabase	Managed PostgreSQL
Auth	JWT HMAC-SHA256	Tokens emitidos por Supabase Auth
Testing	Vitest	Unitario e integración

Arquitectura

```
flowchart TD
    Client["Cliente\n(Angular / Android)"]

    subgraph Worker ["Cloudflare Worker"]
        Entry["src/index.ts\nCORS · Routing · Auth middleware"]
        GQL["GraphQL Yoga\n+ Pothos Schema"]
        Auth["src/lib/auth.ts\nverifyJWT HMAC-SHA256"]
        Errors["src/lib/errors.ts\nTraduce errores BD"]
    end

    subgraph Schema ["src/schema/ (Pothos)"]
        UserS["user/"]
        PostS["post/"]
        CommentsS["comment/"]
        SocialS["social/"]
        RecipeS["recipe/"]
        SearchS["search/"]
    end
    end

    subgraph Data ["Datos"]
        Prisma["Prisma 7\n+ Accelerate"]
        PG["PostgreSQL\nSupabase"]
        SupaAuth["Supabase Auth\n(verifica JWT)"]
    end
    end

    Client -->|"POST /graphql\nAuthorization: Bearer JWT"| Entry
    Entry --> Auth
    Auth -->|"userId extraído"| GQL
    GQL --> Schema
    Schema --> Prisma
    Prisma --> PG
    Auth -.→|"Verifica firma"| SupaAuth
```

Estructura de carpetas

```
src/
├─ index.ts          Entry point: CORS, /graphql, /health, contexto, JWT, errores
├─ lib/
│   ├─ auth.ts       Verificación JWT con HMAC-SHA256
│   ├─ prisma.ts      Instancia y configuración de Prisma
│   └─ errors.ts      Clasificación y traducción de errores de BD
├─ schema/           Schema GraphQL code-first con Pothos
│   ├─ user/         Tipos y resolvers de usuario
│   ├─ post/         Feed, posts, media
│   ├─ comment/      Comentarios
│   ├─ social/       Follow, save, like
│   ├─ recipe/       Recetas + generación IA
│   └─ search/       Búsqueda full-text
└─ services/         Servicios de aplicación reutilizables
```

Endpoints

Endpoint	Método	Descripción
<code>/graphql</code>	POST	API GraphQL + playground GraphiQL
<code>/health</code>	GET	<code>{ status: "ok", timestamp: "..."} </code>

Ejemplo curl

```
curl https://develop.app.savorealo.com/api/graphql \
  -H "Content-Type: application/json" \
  -H "Authorization: Bearer <token>" \
  -d '{"query":{"feed(first:5){ edges{ node{ id title } } } }"}
```

Autenticación

```
sequenceDiagram
    participant C as Cliente
    participant W as Worker
    participant S as Supabase Auth

    C->>W: POST /graphql + Authorization: Bearer <JWT>
    W->>W: Extrae JWT del header
    W->>W: verifyJWT(token, SUPABASE_JWT_SECRET)
    Note over W: HMAC-SHA256 - sin llamada externa
    W->>W: Extrae { userId, email } del payload
    W-->>C: Respuesta GraphQL (userId en contexto)

    Note over W,S: En producción ENVIRONMENT ≠ development → auth obligatoria
```

En `ENVIRONMENT=development` la autenticación es opcional y se usa un usuario por defecto.

Variables de entorno

`.dev.vars` (Wrangler)

```
DATABASE_URL=prisma+postgres://accelerate.prisma-data.net/?api_key=TU_KEY
SUPABASE_JWT_SECRET=tu-jwt-secret
ENVIRONMENT=development
```

.env (Prisma CLI)

```
DATABASE_URL=postgresql://user:pass@pooler.supabase.com:6543/postgres?pgbouncer
DIRECT_URL=postgresql://user:pass@pooler.supabase.com:5432/postgres
```

Variable	Uso
<code>DATABASE_URL</code>	Conexión del Worker (o Hyperdrive si está configurado)
<code>DIRECT_URL</code>	Conexión directa para Prisma CLI y migraciones
<code>SUPABASE_JWT_SECRET</code>	Secreto HMAC para verificar tokens de Supabase Auth
<code>ENVIRONMENT</code>	En producción exige usuario autenticado en todas las operaciones

Operaciones GraphQL

Usuarios

Operación	Tipo	Descripción
<code>user(id)</code>	Query	Perfil por ID. Respeta privacidad
<code>suggestedUsers(preferenceIds, limit)</code>	Query	Usuarios con preferencias similares
<code>checkUsername(username)</code>	Query	Comprueba disponibilidad
<code>searchUsers(query)</code>	Query	Búsqueda por nombre/username
<code>updateProfile(...)</code>	Mutation	Actualiza perfil personal o de negocio

```
# Obtener perfil
query {
  user(id: "uuid") {
    id username display_name avatar_url user_type
    followers_count following_count posts_count
    isFollowing allergies savedPostsCount
  }
}

# Actualizar perfil (todos los campos opcionales)
mutation {
  updateProfile(username: "nuevo", bio: "Amante de la cocina", location: "Madrid") {
    id username display_name
  }
}
```

Feed y Posts

```

flowchart LR
    subgraph Feeds
        F["feed\nSiguiendo · últimos 7d\nRelay pagination"]
        D["discoverFeed\nPersonalizado\nAlgoritmo scoring"]
        U["userPosts\nGrid de perfil\nOffset pagination"]
    end

    subgraph Mutations
        C["createPost\ntitle · content · imageUrl"]
        Del["deletePost\nsolo propios"]
        L["toggleLike\nnda / quita like"]
        S["toggleSave\nguarda / elimina"]
    end

```

Operación	Tipo	Paginación	Descripción
<code>feed(first, after)</code>	Query	Relay cursores	Posts de seguidos, últimos 7 días
<code>discoverFeed(limit, offset)</code>	Query	Offset	Feed personalizado con scoring
<code>userPosts(userId, limit, offset)</code>	Query	Offset	Grid de perfil de usuario
<code>createPost(title, content, imageUrl)</code>	Mutation	—	<code>post_type</code> auto: PHOTO si hay imagen
<code>deletePost(postId)</code>	Mutation	—	Solo propios
<code>toggleLike(postId)</code>	Mutation	—	Alterna like ↔ no like
<code>toggleSave(postId)</code>	Mutation	—	Alterna guardado ↔ no guardado


```
# Feed con paginación Relay
query {
  feed(first: 10, after: "cursor") {
    edges {
      cursor
      node {
        id title description post_type
        likes_count comments_count saves_count
        liked saved imageUrl created_at
        author { id username avatar_url }
        recipe { name time_required difficulty }
      }
    }
    pageInfo { hasNextPage endCursor }
  }
}
```

Comentarios

Operación	Tipo	Descripción
<code>comments(postId)</code>	Query	Lista comentarios (cronológico)
<code>addComment(postId, text)</code>	Mutation	Añadir comentario
<code>deleteComment(commentId)</code>	Mutation	Eliminar (solo propios)

Social

Operación	Tipo	Descripción
<code>toggleFollow(userId)</code>	Mutation	Seguir / dejar de seguir
<code>respondFollowRequest(requestId, accept)</code>	Mutation	Acepta o rechaza solicitud
<code>notifications(limit, offset)</code>	Query	Notificaciones del usuario
<code>pendingFollowRequests</code>	Query	Solicitudes pendientes de aceptar

`toggleFollow` lanza error `Cannot follow yourself` si el `userId` es el propio.

Recetas e IA

```
sequenceDiagram
    participant U as Usuario
    participant W as Worker
    participant AI as Agente IA (Webhook)
    participant DB as PostgreSQL

    U->>W: mutation generateRecipe(message: "...")
    W->>DB: INSERT ai_generations { status: PENDING }
    W->>AI: Webhook asíncrono (fire-and-forget)
    W-->>U: { started: true }
    Note over W,U: Respuesta inmediata
    AI->>AI: Procesa prompt con LLM
    AI->>DB: UPDATE ai_generations { status: DONE, result: {...} }
    U->>W: query myGenerations
    W->>DB: SELECT WHERE userId AND status = DONE
    W-->>U: Recetas generadas
    U->>W: mutation createRecipePost(generationId)
    W-->>U: Post publicado en feed
```

CAMPOS DE **RECIPE** (ANIDADO EN POSTS)

Campo	Tipo	Descripción
<code>id</code>	ID	UUID
<code>name</code>	String	Nombre de la receta
<code>description</code>	String?	Descripción libre
<code>steps</code>	String	JSON stringificado con los pasos
<code>time_required</code>	Int?	Minutos de preparación
<code>servings</code>	Int?	Número de porciones
<code>difficulty</code>	String	<code>EASY</code> · <code>MEDIUM</code> · <code>HARD</code>
<code>estimated_cost</code>	String?	Coste estimado

Búsqueda

Operación	Tipo	Motor	Descripción
<code>searchPosts(query, limit, offset)</code>	Query	<code>tsvector</code> PostgreSQL	Full-text en título y descripción
<code>searchRecipes(query, limit, offset)</code>	Query	<code>search_vector</code>	Full-text en tabla <code>recipes</code>
<code>searchUsers(query)</code>	Query	ILIKE	Por nombre y username

La búsqueda full-text usa diccionario **español** y ordena por relevancia `ts_rank` .

Enums de la base de datos

Enum	Valores
UserType	PERSON · RESTAURANT · BAR
PostType	PHOTO · VIDEO · TEXT · RECIPE
DifficultyLevel	EASY · MEDIUM · HARD
NotificationType	LIKE · COMMENT · FOLLOW · MENTION · RECIPE_SAVE
ConversationType	DIRECT · GROUP
AiGenerationStatus	PENDING · DONE · FAILED
PostCategory	TRENDING · ITALIAN · MEXICAN · JAPANESE · CHINESE · DESSERTS · VEGAN · QUICK_EASY · BURGER · SEAFOOD · COCKTAILS · BREAKFAST · LUNCH · DINNER · SNACKS · HEALTHY · COMFORT_FOOD · STREET_FOOD

Tablas pendientes de implementar

Tabla	Descripción	Prioridad
<code>user_settings</code>	Privacidad, idioma, tema, notificaciones	Alta
<code>notifications</code>	Like, comment, follow, mention	Alta
<code>saved_posts</code> (query)	Listar guardados (toggleSave ya existe)	Alta
<code>viewed_posts</code>	Tracking para discover feed	Alta
<code>stories</code> / <code>viewed_stories</code>	Stories temporales con expiración	Media
<code>conversations</code> / <code>direct_messages</code>	Mensajería directa y grupal	Media
<code>places</code> / <code>place_reviews</code>	Restaurantes, bares y reseñas	Media
<code>ingredients</code> / <code>recipe_ingredients</code>	Ingredientes detallados	Media
<code>contacts</code>	Lista de contactos	Baja

Manejo de errores

`src/lib/errors.ts` intercepta errores de Prisma/PostgreSQL y los transforma en errores GraphQL controlados. Nunca se exponen mensajes técnicos de base de datos al cliente.

```
// Ejemplo: unique constraint → mensaje amigable
PrismaClientKnownRequestError P2002 → GraphQLError("USERNAME_TAKEN")
PrismaClientKnownRequestError P2025 → GraphQLError("NOT_FOUND")
```